

گزارش فنی و تحلیلی پیاده سازی الگوریتم های جستجوی پروژه

«بازی تاج و تخت: بازگشت پادشاه شب»

درس: هوش مصنوعی

استاد: دکتر فریا نصیری مفخم

شماره گروه: ۱۸

اعضای گروه: فرنوش پورشبان، یگانه رستگاری کوپائی، فرزانه سلیمی

پیش گفتار:

در این پروژه، در نقش «آریا استارک»، مأموریت داریم تا پیش از وقوع فاجعه ای دیگر در شمال، تمامی قطعات «شیشه اژدها» (Dragon Glass) را در سرزمین های یخی جمع آوری کنیم. محیط این بازی یک شبکه (Grid) پویاست که چالش های متعددی نظیر زمین های یخی با هزینه های حرکتی بالا، موانع فیزیکی و از همه مهم تر، مسیر متحرک و تهدیدآمیز «پادشاه شب» را در خود جای داده است. چالش اصلی پروژه، یافتن بهینه ترین مسیر برای جمع آوری تمامی قطعات و خروج از محیط، با در نظر گرفتن محدودیت های انرژی و مخاطرات جانی است.

لینک گیت هاب پروژه:

<https://github.com/Farnoush2005/ai-search-project-ui>

۱. پیاده سازی الگوریتم جستجوی اول سطح (BFS)

الگوریتم BFS یک الگوریتم جستجوی ناآگاهانه است که درخت جستجو را سطح به سطح بررسی می کند. این الگوریتم تضمین می کند مسیری با کمترین تعداد گام ها را برای ما پیدا کند و در صورتی که هزینه گام ها تقریباً یکسان باشد، مسیر پیدا شده بهترین مسیر خواهد بود؛ اما به طور کلی، این الگوریتم هزینه مسیر را در نظر نمی گیرد و توجه چندانی به آن ندارد.

```

1  from collections import deque
2
3  def bfs(start_state):
4      # صف و مجموعه استیت‌های ویزیت‌شده رو با نقطه شروع مقداردهی می‌کنیم
5      queue = deque([(start_state, [])])
6      visited = {start_state}
7
8      while queue:
9          curr_state, path = queue.popleft()
10
11         # اگه رسیدیم به هدف، مسیر رو برگردون
12         if curr_state.is_goal_state():
13             return path
14
15         # بررسی حرکتهای مجاز بعدی
16         for action, step_cost, next_state in curr_state.get_successors(toward_walls=True):
17
18             # بی‌خیال مسیریایی که به دیوار ختم میشن
19             if next_state.is_collision_state():
20                 continue
21
22             # اگه این استیت جدید، به صف اضافه میشه تا بعداً چکش کنیم
23             if next_state not in visited:
24                 visited.add(next_state)
25                 queue.append((next_state, path + [action]))
26
27     return [] # اگه هیچ مسیری پیدا نشد

```

ابتدا، برای پیاده‌سازی صف گره‌ها و بهینه‌سازی فرآیند حذف کردن، شی deque را از کتابخانه Collections import می‌کنیم. از آن جایی که deque عملیات حذف و اضافه متد FIFO (First In First Out) را در زمان $O(1)$ انجام می‌دهد، برای کارایی الگوریتم اهمیت زیادی دارد.

تابع BFS را تعریف می‌کنیم. برای جلوگیری از بازسازی معکوس مسیر که هزینه و زمان زیادی به ما تحمیل می‌کند، وضعیت فعلی و مسیر را ذخیره می‌کنیم؛ به این صورت که هر گره در صف به شکل یک تاپل (وضعیت فعلی، مسیر طی شده تا الان) در صف نشان داده می‌شود. بنابراین در قدم اول، صف (queue) خالی با ساختار (start_state, []) مقداردهی اولیه می‌شود.

مجموعه seen را نیز تعریف می‌کنیم و در آن تمام گره‌های پیمایش شده درخت ذخیره می‌شوند تا از تکرار وضعیت یا گیر کردن در لوپ بی‌نهایت جلوگیری کنیم. حلقه while را تعریف می‌کنیم که تا زمانی که صف queue تمام شود یا به جواب برسیم اجرا می‌شود. در هر تکرار، اولین عنصر از صف حذف می‌شود تا تاپل آن بررسی شود. curr_state وضعیت فعلی و path مسیر طی شده تا الان را ذخیره می‌کند. سپس وضعیت فعلی با شرط و تابع is_goal_state() سنجیده می‌شود و در صورتی که وضعیت هدف ما باشد، یعنی جمع‌آوری تمام شیشه‌های اژدها، path مربوطه به عنوان خروجی بر می‌گردانده می‌شود.

در غیر این صورت، حلقه for را برای بررسی همسایه‌ها تشکیل می‌دهیم که با متد get_successors(toward_walls=True) فراخوانی می‌شوند. این متد لیستی از تاپل‌ها را بر می‌گرداند که شامل action (حرکت/کنش)، step_cost (در این تابع استفاده نمی‌شود) و next_state (وضعیت بعدی) است.

استفاده از `toward_wall=True` به این دلیل است که می‌خواهیم وضعیت‌های بعدی که شامل حرکت به سمت جعبه‌ها می‌شوند را نیز در نظر بگیرد چرا که برخورد با مانع نباید باعث توقف جستجو شود بلکه عامل ما باید صرفاً تغییر مسیر دهد.

اگر در وضعیت بعدی (`next_state`) تابع `is_collision_state` برقرار باشد، بدین معناست که جان اسنو (عامل) با پادشاه شب برخورد کرده است. در این حالت گره مربوطه به صف اضافه نمی‌شود، آن شاخه از جستجو حذف می‌شود و سراغ وضعیت بعدی دیگر می‌رویم. همچنین اگر وضعیت بعدی در مجموعه `seen` نیست، آنگاه آن را به مجموعه `seen` و سپس به صف اضافه می‌کنیم.

۲. پیاده‌سازی جستجوی عمق محدود (DLS)

الگوریتم DLS نسخه‌ای کنترل‌شده از DFS است که برای حل مشکل حافظه و چرخه‌های بی‌انتهای در محیط‌های بزرگ، یک سقف یا آستانه برای عمق جستجو (`limit`) تعیین می‌کند. این الگوریتم برای محیط‌های کوچک و سبک (مانند نقشه‌های سطح Easy بدون پادشاه شب) بسیار سریع عمل می‌کند.

```
1 def dls(start_state):
2     # می‌خوره easy سبک و سریع که بیشتر به درد می‌ماید DLS به
3     if start_state.is_goal_state():
4         return []
5
6     # تابع بازگشتی برای پیمایش تا عمق مشخص
7     def recursive_dls(curr_state, limit, path):
8         if curr_state.is_goal_state():
9             return path
10
11        # رسیدیم به لیمیت، دیگه پایین‌تر نمی‌ریم
12        if limit <= 0:
13            return None
14
15        # چک کردن حرکتهای بعدی
16        for action, step_cost, next_state in curr_state.get_successors():
17            result = recursive_dls(next_state, limit - 1, path + [action])
18            if result is not None:
19                return result
20
21        return None
22
23    # عمق رو کم در نظر گرفتیم
24    depth_limit = 18
25
26    result = recursive_dls(start_state, depth_limit, [])
27
28    if result:
29        print(f"DLS done with depth : {depth_limit}")
30        return result
31    else:
32        print(f"DLS not done with :{depth_limit} ")
33        return []
34
35    # برای استفاده تو main.py
36    dlsa = dls
```

تابع را با وضعیت شروع به عنوان ورودی تعریف می‌کنیم. در صورتی که وضعیت اولیه، وضعیت هدف ما بود مسیر خالی برگردانده می‌شود و نیازی به حرکت نیست. در غیر این صورت، تابع `recursive_dls` را درون این تابع تعریف

می‌کنیم که وضعیت فعلی (`curr_state`)، حداکثر عمق مجاز برای جستجو (`limit`) و مسیر طی شده (`path`) را به عنوان ورودی دریافت می‌کند. اگر `curr_state` وضعیت هدف بود، تابع مسیر آن را برمی‌گرداند. اگر عمق جستجو کمتر از صفر بود نیز `None` برگردانده می‌شود که یعنی جستجو شاخه بی نتیجه بوده.

در حلقه `for` با متد `get_successors()` وضعیت‌های بعد فراخوانی می‌شوند. هر خروجی متد شامل `action` (کنش)، `step_cost` (هزینه که در این الگوریتم در نظر گرفته نمی‌شود) و `next_state` (وضعیت بعدی) است.

برای هر وضعیت بعدی، تابع خودش را صدا می‌زند، با این تفاوت که هر بار از عمق یکی کم می‌شود و کنش جدید به مسیر اضافه می‌شود. در واقع تابع درونی تا زمانی که متغیر `limit` به صفر نرسیده به صورت عمقی (`DFS`)، به پیشروی در درخت حالات ادامه می‌دهد.

اگر یکی از شاخه‌ها به هدف برسد (یعنی `result` برابر `none` نباشد)، همان جواب برگردانده می‌شود و جستجو متوقف می‌شود و اگر هیچ شاخه‌ای به نتیجه نرسید، `none` برگردانده می‌شوند.

پس از اجرای کد با عمق‌های مختلف و بررسی نتایج، مناسب‌ترین عمق جستجو برای شیشه‌های اژدها ۱۸ انتخاب شد تا محاسبات در کم‌ترین زمان و بیشترین سرعت انجام شوند.

جستجو از حالت شروع با مسیر خالی آغاز می‌شود و نتیجه‌ی آن در `result` ذخیره می‌شود. اگر `result` وجود داشته باشد، عمقی که در آن پیدا شده نمایش داده می‌شود و مسیر طی شده بازگردانده می‌شود. در غیر این صورت مسیری وجود ندارد و یک لیست خالی برگردانده می‌شود.

همچنین باید در انتها ذکر کنیم که در اجرای این کد، زمان زیادی برای پردازش و ران شدن سپری شد.

۳. پیاده‌سازی الگوریتم جستجوی هزینه یکنواخت (UCS)

با توجه به اینکه خانه‌های مختلف محیط (مانند صخره‌های یخی) هزینه‌های متفاوتی دارند، `BFS` دیگر بهینه نیست و `UCS` با بسط دادن کم‌هزینه‌ترین گره‌ها، بهینگی مسیر را تضمین می‌کند. الگوریتم `UCS` یک الگوریتم جستجوی ناآگاهانه است که به جای سطح‌بندی (مانند `BFS`)، وضعیت‌ها را بر اساس کمترین هزینه تجمعی توسعه می‌دهد. برخلاف `BFS` که فقط تعداد گام‌ها را می‌شمارد، `UCS` با در نظر گرفتن وزن یال‌ها (`Step Cost`)، برای مسائلی که هزینه‌های متفاوت دارند، ایده‌آل است.

```

1 import heapq
2 import itertools
3
4 def ucs(start_state):
5     queue = []
6     counter = itertools.count() # ها تو صف Node برای جلوگیری از ارور مقایسه
7
8     heapq.heappush(queue, (0, next(counter), start_state, []))
9
10    # مپ کردن استیتها به کمترین هزینه دیده شده تا الان
11    visited = {start_state: 0}
12
13    while queue:
14        cost, _, curr_state, path = heapq.heappop(queue)
15
16        # اگه رسیدیم به هدف، مسیر رو برگردون
17        if curr_state.is_goal_state():
18            return path
19
20        # بررسی حرکتهای مجاز بعدی
21        for action, step_cost, next_state in curr_state.get_successors(toward_walls=True):
22
23            # بیخیال مسیری که به دیوار ختم میشن
24            if next_state.is_collision_state():
25                continue
26
27            new_cost = cost + step_cost
28
29            # اگه استیت جدید با مسیر کم هزینه تری براش پیدا کردیم، به صف اضافه اش می کنیم
30            if next_state not in visited or new_cost < visited[next_state]:
31                visited[next_state] = new_cost
32                heapq.heappush(queue, (new_cost, next(counter), next_state, path + [action]))
33
34    return [] # اگه هیچ مسیری پیدا نشد

```

برای پیاده سازی این الگوریتم، ابتدا کتابخانه های `heapq` برای پیاده سازی صف اولویت و `itertools` برای مدیریت تساوی هزینه ها `import` می شوند. از آن جایی که `heapq` در پایتون یک "Min-Heap" است، همیشه گره ای با کمترین هزینه در ریشه قرار می گیرد و با زمان اجرایی معادل ($O(\log n)$) برای عملیات حذف و اضافه، بسیار بهینه عمل می کند.

تابع UCS را با وضعیت شروع به عنوان ورودی تعریف می کنیم و یک صف خالی `queue` تعریف می کنیم. برای اطلاعات جهت جلوگیری از مقایسه مستقیم دیتای وضعیت ها، در صورت برابر بودن هزینه ها (و در نتیجه جلوگیری از خطای `TypeError`)، از یک شمارنده (`counter`) استفاده می کنیم. در صف، هر گره به صورت تاپل `(cost, tie_breaker, path, state)` ذخیره می شود. بنابراین در ابتدای کار، صف با مقدار زیر مقداردهی اولیه می کنیم:

```
(queue, (0, next(counter), start_state, []))
```

دیکشنری `visited` برای ذخیره و ردیابی کمترین هزینه رسیدن به هر وضعیت تعریف شده تا از دوباره کاری و بازگشت به وضعیت هایی که قبلاً با هزینه کمتری پیموده شده اند، جلوگیری شود. حلقه `while` نیز وظیفه جستجو را بر عهده دارد و تا زمانی که به جواب برسیم یا صف خالی شود، اجرا می شود. در هر تکرار، `heapq.heappop` گره ای با کمترین هزینه را از صف استخراج می کند و اطلاعات آن را در ذخیره می کند تا بررسی شود. ابتدا با متد `is_goal_state()`

بررسی می‌کنیم که آیا وضعیت فعلی همان وضعیت هدف مورد نظر است یا خیر؛ در صورت تایید، مسیر تا آن لحظه برگردانده می‌شود.

در غیر این صورت، حلقه‌ی for از طریق متد `get_successors(toward_walls=True)` تمام کنش‌های ممکن و وضعیت‌های بعدی را استخراج می‌کند. خروجی آن خروجی متد شامل `action` (کنش)، `step_cost` (هزینه‌ی حرکت) و `next_state` (وضعیت بعدی) است. متغیر `step_cost` در اینجا به هزینه تجمعی فعلی (`cost`) اضافه می‌شود تا `new_cost` محاسبه گردد. اگر وضعیت بعدی با استفاده از `is_collision_state()` منجر به برخورد با پادشاه شب شود، مانند قبل آن شاخه از جستجو حذف شده و با دستور `continue` نادیده گرفته می‌شود.

در شرط نهایی بررسی می‌کنیم اگر وضعیت بعدی تا به حال پیمایش نشده یا مسیر جدیدی با هزینه کمتری به آن وضعیت پیدا شده، آن را در دیکشنری `visited` ثبت و به صف اضافه کند. با داشتن این شرط از پیدا کردن مسیر بهینه اطمینان داریم.

در نهایت، اگر صف خالی شود و به هدف نرسیم، تابع لیستی خالی باز می‌گرداند که به معنای عدم وجود مسیر منتهی به هدف است.

۴. پیاده‌سازی الگوریتم آگاهانه A^* به همراه تابع (Heuristic)

الگوریتم جستجوی A^* (A Star) یکی از معروف‌ترین الگوریتم‌های جستجو در حوزه هوش مصنوعی محسوب می‌شود. این الگوریتم که در دسته جستجوهای ناآگاهانه قرار می‌گیرد، قادر است بهینه‌ترین مسیر را از یک وضعیت آغازین به یک وضعیت هدف در یک گراف پیدا کند. A^* با بهره‌گیری هوشمندانه از اطلاعات هزینه طی شده و همچنین تخمین هزینه باقی‌مانده تا هدف، کارایی بیشتری نسبت به الگوریتم‌های مبتنی بر هزینه یا تخمین دارد. ماهیت `Admissible` و `Consistent` بودن این الگوریتم، تضمین می‌کند که کوتاه‌ترین و بهینه‌ترین مسیر ممکن یافت خواهد شد.

در ادامه به بررسی و تحلیل کدهای این بخش می‌پردازیم.

```

1 import heapq
2 import itertools
3
4 def astar(start_state):
5     # تابع هیوریستیک برای تخمین هزینه باقی‌مانده تا هدف
6     def get_h(state):
7         targets = list(state.get_targets_positions())
8         if not targets:
9             return 0
10
11         pos = state.get_agent_position()
12         w_pos = state.get_weapon_position()
13         has_w = state.has_weapon()
14
15         # فاصله منتهن تا نزدیکترین هدف
16         dist_to_t = min(abs(pos[0] - t[0]) + abs(pos[1] - t[1]) for t in targets)
17
18         # اگر چندان هدف داشتیم MST محاسبه
19         mst = 0
20         if len(targets) > 1:
21             edges = []
22             for i in range(len(targets)):
23                 for j in range(i + 1, len(targets)):
24                     d = abs(targets[i][0] - targets[j][0]) + abs(targets[i][1] - targets[j][1])
25                     edges.append((d, i, j))
26             edges.sort()
27
28             # پیاده‌سازی کروسکال برای درخت پوشای کمینه
29             parent = list(range(len(targets)))
30             def find(i):
31                 if parent[i] == i:
32                     return i
33                 parent[i] = find(parent[i]) # path compression
34             return parent[i]
35
36             for d, u, v in edges:
37                 r1, r2 = find(u), find(v)
38                 if r1 != r2:
39                     parent[r1] = r2
40                     mst += d
41

```

```

42     # هزینه تخمینی بدون در نظر گرفتن سلاح
43     c_no_w = (dist_to_t + mst) * 5
44
45     # اگر اسلحه هست ولی نداریمش، چک می‌کنیم رفتن سزاغش می‌ارزه یا نه
46     if w_pos and not has_w:
47         d_w = abs(pos[0] - w_pos[0]) + abs(pos[1] - w_pos[1])
48         d_w_t = min(abs(w_pos[0] - t[0]) + abs(w_pos[1] - t[1]) for t in targets)
49         return min(c_no_w, (d_w + d_w_t + mst) * 5)
50
51     return c_no_w
52
53     # A* صف اولویت برای الگوریتم
54     queue = []
55     counter = itertools.count() # Node‌ها برای جلوگیری از آزار متناهی
56     heapq.heappush(queue, (get_h(start_state), 0, next(counter), start_state, []))
57
58     # یپ کردن استیت‌ها به کمترین هزینه دیده‌شده
59     visited = {}
60
61     while queue:
62         f_cost, cost, _, curr_state, path = heapq.heappop(queue)
63
64         # رسیدن به هدف
65         if curr_state.is_goal_state():
66             return path
67
68         # بررسی تکراری نبودن استیت با هزینه بیشتر
69         if curr_state in visited and visited[curr_state] <= cost:
70             continue
71         visited[curr_state] = cost
72
73         # بررسی حرکتهای بعدی
74         for action, step_cost, next_state in curr_state.get_successors(toward_walls=True):
75             # برخورد با دیوار مجاز نیست، مگر اینکه خود هدف باشه
76             if next_state.is_collision_state() and not next_state.is_goal_state():
77                 continue
78
79             new_cost = cost + step_cost
80

```

```

81     # اگر مسیر بهتری پیدا کردیم، استیت جدید رو اضافه می‌کنیم
82     if next_state not in visited or new_cost < visited[next_state]:
83         h = get_h(next_state)
84         heapq.heappush(queue, (new_cost + h, new_cost, next(counter), next_state, path + [action]))
85
86     return [] # اگر هیچ مسیری پیدا نشد

```

تابع `astar` با ورودی `start_state` تعریف می‌شود. در ابتدا، تابع داخلی `get_h(state)` تعریف می‌شود که محاسبات هیوریستیک را انجام می‌دهد. این تابع با دریافت موقعیت اهداف با متد `get_target_postitions`، موقعیت فعلی عامل با متد `get_agent_positions` و وضعیت سلاح با متد `has_weapon()`، هزینه تخمینی باقی‌مانده را محاسبه می‌کند. ابتدا، با متغیر `dist_to_t` فاصله منتهی تا نزدیک‌ترین هدف محاسبه می‌شود.

در شرط بررسی می‌کنیم، در صورتی که بیش از یک هدف وجود داشته باشد، برای تخمین بهتر هزینه اتصال تمامی اهداف به یکدیگر، از الگوریتم کروسکال استفاده می‌کنیم تا بتوانیم درخت پوشای کمینه (MST) را پیدا کنیم. اگر سلاحی در محیط وجود داشته باشد و عامل آن را نداشته باشد، هیوریستیک دو حالت را در نظر می‌گیرد. اینکه بدون برداشتن سلاح به سمت اهداف برود یا پس از برداشتن سلاح به سمت اهداف برود.

تابع با کمینه گرفتن بین این دو حالت $\min(c_no_w, (d_w + d_w_t + mst) * 5)$ ، تعیین می‌کند که انحراف از مسیر برای برداشتن سلاح از نظر هزینه نهایی به صرفه است یا خیر. سپس صف اولویت (queue) مقداردهی اولیه می‌شود. این صف، حالت‌های موجود در صف را بر اساس مقدار $f(n)$ مرتب می‌سازد؛ کمترین مقدار f همواره بالاترین اولویت را دارد. برای جلوگیری از بروز خطاهای مقایسه‌ای هنگام برخورد با دو حالت یکسان از نظر هزینه، از یک شمارنده (`itertools.count()`) به عنوان عامل تفکیک‌کننده استفاده می‌شود. اولین عنصر صف شامل هزینه کلی (f_cost) حالت شروع، هزینه طی شده ($cost$ که صفر است)، مقدار شمارنده، حالت شروع (`start_state`) و مسیر خالی (`[]`) است.

دیکشنری `visited` نیز برای ثبت حالت‌های پردازش شده ایجاد می‌شود تا از حالت‌های تکراری و جلوگیری از ورود به حلقه‌های بی‌نهایت اطمینان حاصل می‌کند. حلقه `while` تا زمانی که صف اولویت خالی نشده باشد، ادامه می‌یابد. در هر تکرار، حالتی با کمترین f_cost از صف خارج می‌شود. سپس بررسی می‌شود که آیا این وضعیت، وضعیت هدف است یا خیر. اگر چنین بود، مسیر (`path`) بازگردانده شده و جستجو پایان می‌یابد. در غیر این صورت، در شرطی بررسی می‌شود که آیا این حالت قبلاً با هزینه‌ای کمتر یا مساوی پردازش شده است. اگر پاسخ درست بود، از آن عبور می‌شود تا از پردازش تکراری جلوگیری شود. در غیر این صورت، کمترین هزینه رسیدن به این حالت در `visited` ثبت می‌گردد. سپس، تابع `get_successors(toward_walls=True)` فراخوانی می‌شود تا تمامی وضعیت‌های ممکن بعدی به همراه حرکت مربوطه و هزینه آن تولید شوند.

در این مرحله، وضعیت‌هایی که منجر به برخورد با موانع (مانند دیوارها) می‌شوند، فیلتر می‌شوند، مگر اینکه خود آن مانع، حالت هدف باشد. برای هر وضعیت بعدی معتبر، هزینه جدید (`new_cost`) محاسبه شده و هیوریستیک آن (h) نیز به دست می‌آید. اگر وضعیت جانشین هنوز دیده نشده باشد یا مسیر جدید به آن کوتاه‌تر از مسیر قبلی باشد، این وضعیت جدید به همراه f_cost به‌روز شده‌اش به صف اولویت اضافه می‌شود. در نهایت، اگر حلقه `while` به اتمام برسد و هدف یافت نشود، به این معنی است که هیچ مسیری وجود ندارد و تابع یک لیست خالی بازمی‌گرداند.

نتیجه نهایی در اجرا:

بهینه‌سازی: استفاده از هیوریستیک MST در A^* ، باعث کاهش چشمگیر گره‌های بسط داده شده (Expanded Nodes) و تضمین بهینگی مسیر شده است.

تحلیل الگوریتم‌ها (خلاصه)

• BFS: یافتن کوتاه‌ترین مسیر (از نظر تعداد گام) بدون توجه به هزینه‌های terrain.

```
=====
● → AI26-PRJ1 /usr/local/bin/python3 /Users/farnoush/Downloads/AI26-PRJ1/main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.4)
Hello from the pygame community. https://www.pygame.org/contribute.html
Map Name: easy-no-weapon
Search completed in 0.03s
Final Score: 720
Cost: 280
Expanded Nodes: 1583
Steps Taken: 18
=====
```

• UCS: یافتن مسیر با کمترین هزینه (ایده‌آل برای نقشه‌های دارای یخ)

```
=====
● → AI26-PRJ1 /usr/local/bin/python3 /Users/farnoush/Downloads/AI26-PRJ1/main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.4)
Hello from the pygame community. https://www.pygame.org/contribute.html
Map Name: easy-no-weapon
Search completed in 0.01s
Final Score: 900
Cost: 100
Expanded Nodes: 704
Steps Taken: 20
=====
```

• A^* : ترکیب بهینه سرعت و دقت با استفاده از هیوریستیک MST (بهترین عملکرد)

```
=====
● → AI26-PRJ1 /usr/local/bin/python3 /Users/farnoush/Downloads/AI26-PRJ1/main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.4)
Hello from the pygame community. https://www.pygame.org/contribute.html
Map Name: easy-no-weapon
Search completed in 0.01s
Final Score: 900
Cost: 100
Expanded Nodes: 93
Steps Taken: 20
=====
```

• DLS: مناسب برای نقشه‌های بسیار ساده و محدودیت عمق مشخص.

```
=====
→ AI26-PRJ1 /usr/local/bin/python3 /Users/farnoush/Downloads/AI26-PRJ1/main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.12.4)
Hello from the pygame community. https://www.pygame.org/contribute.html
DLS done with depth : 18
Map Name: easy-no-weapon
Search completed in 121.63s
Final Score: 720
Cost: 280
Expanded Nodes: 24853614
Steps Taken: 18
=====
```